

فاز ①،

در این فاز ساختار فیلد ۱۲۸، را برپایه ساز می‌کرده‌ایم. نتایج استفاده شده به شرح زیر است :

```
> def sbox_output(sbox, inp: int): ...  
> def pre_key_1(x: bytes): ...  
> def pre_key_2(x: bytes): ...  
> def key_gen_1(x: bytes): ...  
> def key_gen_2(x: bytes): ...  
> def key_gen_3(x: bytes): ...  
> def key_gen_4(x: bytes): ...  
> def key_gen(x: bytes): ...  
> def round_function(x: bytes, key: bytes) -> bytes: ...  
> def encrypt(m: bytes, k: bytes) -> bytes: ...  
> def decrypt(m: bytes, k: bytes) -> bytes: ...  
> def expand_and_split(data: bytes, block_size: int = 16) -> tuple[list[bytes], int]: ...  
> def unpad_message(data: bytes, padding_length: int) -> bytes: ...
```

1 - sbox-output ← خروجی نام از sbox مورد نظر را می‌دهد.

pre-key_i-2
Key-gen_i-3
Key-gen-4
تولید کلید

round₅-function ← تابع f

6 - decrypt/encrypt ← رمزنگاری و رمزگشایی

7 - expand and split ← padding پیام را قطعه‌قطعه کردن به بلوک‌های ۱۲۸ بیتی

8 - unpad ← تبدیل پیام به همان اندازه اصلی خودش

9 - نتایج مُدهای مختلف

در ادامه به بررسی دقیق‌تر هر یک می‌پردازیم :

1- تولید کلید : با توجه به ماژول تولید کلید ابتدا باید دو ماژول از نوع preKey و Key-gen-i تولید کنیم. پس با توجه به شکل ماژول خروجی‌های مناسب هر کدام را به ورودی دیگری وصل کنیم. تجزیه این ماژول‌ها در تابع Key-gen انجام شده است که خروجی آن لیست از کلیدهاست (طول لیست = ۱۶)

```
def key_gen_1(x: bytes):
    if len(x) != 16:
        raise ValueError("Input x must be 16 bytes (128 bits).")

    K_a = (
        sbbox_output(Sbox5, x[8]) ^
        sbbox_output(Sbox6, x[9]) ^
        sbbox_output(Sbox7, x[7]) ^
        sbbox_output(Sbox8, x[6]) ^
        sbbox_output(Sbox5, x[2])
    )
    K_b = (
        sbbox_output(Sbox5, x[10]) ^
        sbbox_output(Sbox6, x[11]) ^
        sbbox_output(Sbox7, x[5]) ^
        sbbox_output(Sbox8, x[4]) ^
        sbbox_output(Sbox6, x[6])
    )
    K1 = ((K_a & 0xFFFFFFFF) << 32) | (K_b & 0xFFFFFFFF)

    K_c = (
        sbbox_output(Sbox5, x[12]) ^
        sbbox_output(Sbox6, x[13]) ^
        sbbox_output(Sbox7, x[3]) ^
        sbbox_output(Sbox8, x[2]) ^
        sbbox_output(Sbox7, x[9])
    )
    K_d = (
        sbbox_output(Sbox5, x[14]) ^
        sbbox_output(Sbox6, x[15]) ^
        sbbox_output(Sbox7, x[1]) ^
        sbbox_output(Sbox8, x[0]) ^
        sbbox_output(Sbox8, x[13])
    )
    K2 = ((K_c & 0xFFFFFFFF) << 32) | (K_d & 0xFFFFFFFF)

    return K1, K2
```

```
# Initialize z as a 16-byte array (128-bit number)
z = [0] * 16

# First operation: z[0:4]
z_part = (
    int.from_bytes(x[0:4], byteorder="big") ^
    sbbox_output(Sbox5, x[13]) ^
    sbbox_output(Sbox6, x[15]) ^
    sbbox_output(Sbox7, x[12]) ^
    sbbox_output(Sbox8, x[14]) ^
    sbbox_output(Sbox7, x[8])
)
z[0:4] = z_part.to_bytes(4, byteorder="big")

# Second operation: z[4:8]
z_part = (
    int.from_bytes(x[8:12], byteorder="big") ^
    sbbox_output(Sbox5, z[0]) ^
    sbbox_output(Sbox6, z[1]) ^
    sbbox_output(Sbox7, z[2]) ^
    sbbox_output(Sbox8, z[3]) ^
    sbbox_output(Sbox8, x[10])
)
z[4:8] = z_part.to_bytes(4, byteorder="big")

# Third operation: z[8:12]
z_part = (
    int.from_bytes(x[12:16], byteorder="big") ^
    sbbox_output(Sbox5, z[4]) ^
    sbbox_output(Sbox6, z[5])
```

قسمت از، پیاده سازی key-gen-1

قسمت از پیاده سازی prekey 1

*) برای اینکه کلیت سروس byte انجام می شود از دایره سخت بایستی بایستی استفاده شده است.

```
def key_gen(x: bytes):
    if len(x) != 16:
        raise ValueError("Input x must be 16 bytes (128 bits).")

    key = [0] * 16

    # Round 1
    z = pre_key_1(x)
    key[0], key[1] = key_gen_1(z)
    x = pre_key_2(z)
    key[2], key[3] = key_gen_2(x)

    # Round 2
    z = pre_key_1(x)
    key[4], key[5] = key_gen_3(z)
    x = pre_key_2(z)
    key[6], key[7] = key_gen_4(x)

    # Round 3
    z = pre_key_1(x)
    key[8], key[9] = key_gen_1(z)
    x = pre_key_2(z)
    key[10], key[11] = key_gen_2(x)

    # Round 4
    z = pre_key_1(x)
    key[12], key[13] = key_gen_3(z)
    x = pre_key_2(z)
    key[14], key[15] = key_gen_4(x)

    return key
```

تجميع مدارهای pre-key 1 و key-gen 1 در تابع key-gen که فروپاشی آن key [1...16] است.

2- تابع دور:

```
def round_function(x: bytes, key: bytes) -> bytes:
    if len(x) != 8 or len(key) != 8:
        raise ValueError("Both x and key must be 8 bytes long.")

    # Compute z array by XORing x and reversed key
    z = [x[i] ^ key[7 - i] for i in range(8)]

    # Apply S-box substitutions
    z[0] = sbox_output(Sbox1, z[0])
    z[1] = sbox_output(Sbox2, z[1])
    z[2] = sbox_output(Sbox3, z[2])
    z[3] = sbox_output(Sbox4, z[3])
    z[4] = sbox_output(Sbox2, z[4])
    z[5] = sbox_output(Sbox3, z[5])
    z[6] = sbox_output(Sbox4, z[6])
    z[7] = sbox_output(Sbox1, z[7])

    # Compute out array using XOR operations
    out = [
        z[7] ^ z[6] ^ z[5] ^ z[3] ^ z[2] ^ z[0],
        z[7] ^ z[6] ^ z[4] ^ z[3] ^ z[1] ^ z[0],
        z[7] ^ z[5] ^ z[4] ^ z[2] ^ z[1] ^ z[0],
        z[6] ^ z[5] ^ z[4] ^ z[3] ^ z[2] ^ z[1],
        z[7] ^ z[6] ^ z[5] ^ z[1] ^ z[0],
        z[7] ^ z[6] ^ z[4] ^ z[2] ^ z[1],
        z[7] ^ z[5] ^ z[4] ^ z[3] ^ z[2],
        z[6] ^ z[5] ^ z[4] ^ z[3] ^ z[0]
    ]

    # Convert the out array into bytes
    return bytes(out)
```

در تابع دور ابتدا بایت های x و key به ترتیب گفته شده XOR شده اند سپس ماتریس مورد نظر فوجی را تولید می کند. ضرب های ماتریس به شکل 8 بایت XOR آورده شده است.

3- تولید رمزگزار در رمزنگاری:

پس از آنکه توابع f و $key\ generator$ آماده شوند می توان ساختار کل رمزگزار و رمزنگار 128 بیت را بسازیم. ابتدا پیام به دو قسمت 64 بیتی تقسیم می شود و در یک f که ترتیب متغیر در رمزگزار و رمزنگار دارند، کلمات XOR و تابع f اعمال می کنیم و در نهایت در آخر فوجی را $return$ می کنیم.

```
def encrypt(m: bytes, k: bytes) -> bytes:
    key = key_gen(k)
    Key_bytes = [key[i].to_bytes(8, byteorder='big') for i in range(16)]
    R = m[:8] # First 8 bytes
    L = m[8:] # Last 8 bytes

    for i in range(16):
        temp_R = R
        temp_L = L
        L = R
        R = int.to_bytes(int.from_bytes(round_function(temp_R, Key_bytes[i]), 'big') ^ int.from_bytes(temp_L, 'big'), 8, 'big')

    return L + R

def decrypt(m: bytes, k: bytes) -> bytes:
    key = key_gen(k)
    Key_bytes = [key[i].to_bytes(8, byteorder='big') for i in range(16)]
    R = m[:8] # First 8 bytes
    L = m[8:] # Last 8 bytes

    for i in range(15, -1, -1):
        temp_R = R
        temp_L = L
        L = R
        R = int.to_bytes(int.from_bytes(round_function(temp_R, Key_bytes[i]), 'big') ^ int.from_bytes(temp_L, 'big'), 8, 'big')

    return L + R
```

تقسیم متن به 2 قسمت

حلقه ی صفوی رمزگذاری

حلقه ی نهایی رمزگشایی

4 - expand and split و unpad :

```
def expand_and_split(data: bytes, block_size: int = 16) -> tuple[list[bytes], int]:  
    padding_length = (-len(data)) % block_size  
    padded_data = data + bytes([0] * padding_length) # Pad with null bytes  
    blocks = [padded_data[i:i + block_size] for i in range(0, len(padded_data), block_size)]  
    return blocks, padding_length
```

در سمت اول این تابع بررسی می‌شود که چند بایت نیاز است که اضافه شود تا
عدد بر block-size که به طور پیش فرض 128 بیت / 16 بایت است تقسیم
شود. بعد از پیدا کردن این عدد به آنجا که بایت‌ها اضافه می‌کنیم.

```
def unpad_message(data: bytes, padding_length: int) -> bytes:  
    # Remove padding_length bytes and then strip any null characters (\x00) from the end  
    return data.rstrip(b'\x00') # Removes any trailing null characters
```

موقعی که می‌خواهیم پیام را به حالت عادی برگردانیم از انتهای پیام صفرها را حذف
می‌کنیم. برای اینکار از متد `rstrip` استفاده می‌کنیم.

5 - مرهای مختلف :

5-1 - EBC : با کمک تابع `expand and split` متد به بلوک‌های
16 بایتی تقسیم می‌کنیم و بسته به اینکه می‌خواهیم رمزگذار یا گشای انجام دهیم

```
def EBC(m: str, k: bytes, enc: bool = True):  
    if enc:  
        # Encode the message as bytes using UTF-8  
        m_bytes = m.encode("utf-8")  
        blocks, padding_length = expand_and_split(m_bytes, block_size=16) # 16 bytes = 128 bits  
    else:  
        # Assume ciphertext is provided as bytes  
        m_bytes = bytes.fromhex(m)  
        (variable) processed_blocks: list and split(m_bytes, block_size=16)  
  
    processed_blocks = []  
    for block in blocks:  
        if enc:  
            processed_blocks.append(encrypt(block, k))  
        else:  
            processed_blocks.append(decrypt(block, k))  
  
    # Combine the processed blocks into a single result  
    result = b"".join(processed_blocks)  
  
    if enc:  
        # Return as hex string for easier storage/transmission  
        return result.hex(), padding_length  
    else:  
        # Remove padding and decode back to the original string  
        result = unpad_message(result, padding_length)  
        return result.decode("utf-8", errors="ignore")
```

توابع `encrypt` و
`decrypt` را فراخوانی
می‌کنیم و در آرایه‌ی
`processed blocks` می‌ریزم.

برای اینکه `string` را تبدیل به
`byte` بکنیم از `utf-8 encode`
استفاده کرده ایم.

```
def OFB(m: str, k: bytes, Nonce: bytes, enc: bool = True):
    if enc:
        m_bytes = m.encode("utf-8")
        blocks, padding_length = expand_and_split(m_bytes, block_size=16) # 16 bytes = 128 bits
    else:
        m_bytes = bytes.fromhex(m)
        blocks, padding_length = expand_and_split(m_bytes, block_size=16)

    input = Nonce
    processed_blocks = []
    for block in blocks:
        if enc:
            # Encrypt the input and XOR it with the current block
            input = encrypt(input, k)
            processed_blocks.append(bytes(a ^ b for a, b in zip(input, block)))
        else:
            # Decrypt the input and XOR it with the current block
            input = encrypt(input, k)
            processed_blocks.append(bytes(a ^ b for a, b in zip(input, block)))

    # Combine the processed blocks into a single result
    result = b"".join(processed_blocks)

    if enc:
        # Return as hex string for easier storage/transmission
        return result.hex(), padding_length
    else:
        # Remove padding and decode back to the original string
        result = unpad_message(result, padding_length)
        return result.decode("utf-8", errors="ignore")
```

2-5- CFB: در ابتدا

تمام مراحل تکرار می‌شود

تا آنجی می‌رسد که به باور

توضیح دارم. تفاوت

تفاوت در قسمت پردازش

بلوک‌ها در کد است.

در encryption، متغیر

input در ابتدا Nonce و

بعد از آن به وسیله خروجی encrypt در decryption هم به این اتفاق

می‌افتد.

3-5- CTR: در هر CTR در یک مرحله هر بار برای encryption و decryption

به ازای هر آن، آن را به عدد 8 بایتی تبدیل کرده و با Nonce، Concat می‌کنیم.

و سپس محاسبه XOR را انجام می‌دهیم و بلوک‌ها را تولید می‌دهیم تا در یک لیست می‌ریزم.

```
def CTR(m: str, k: bytes, Nonce: bytes, enc: bool = True):
    if enc:
        m_bytes = m.encode("utf-8")
        blocks, padding_length = expand_and_split(m_bytes, block_size=16) # 16 bytes = 128 bits
    else:
        m_bytes = bytes.fromhex(m)
        blocks, padding_length = expand_and_split(m_bytes, block_size=16)

    processed_blocks = []

    for i in range(len(blocks)):
        input = Nonce + i.to_bytes(8, byteorder='big')
        encrypted_input = encrypt(input, k)
        if enc:
            # Encrypt the input and XOR it with the current block
            processed_blocks.append(bytes(a ^ b for a, b in zip(encrypted_input, blocks[i])))
        else:
            # Decrypt the input and XOR it with the current block
            processed_blocks.append(bytes(a ^ b for a, b in zip(encrypted_input, blocks[i])))

    # Combine the processed blocks into a single result
    result = b"".join(processed_blocks)

    if enc:
        # Return as hex string for easier storage/transmission
        return result.hex(), padding_length
    else:
        # Remove padding and decode back to the original string
        result = unpad_message(result, padding_length)
        return result.decode("utf-8", errors="ignore")
```

محاسبه رمزنگاری و

رمزگشایی هر دو با

تابع encrypt انجام

می‌شود.

5-4 - CBC: محلیات در CBC واحدی ترکیبی از OFB و CTR است یعنی ورودی XOR خواهد بود سرش به هم مرتبطند و از None هم استفاده می‌کنیم.

البته برای رمزنگاری از encryption و برای رمزگشایی از decryption استفاده می‌کنیم.

```
def CBC(m: str, k: bytes, Nonce: bytes, enc: bool = True):
    if enc:
        m_bytes = m.encode("utf-8")
        blocks, padding_length = expand_and_split(m_bytes, block_size=16) # 16 bytes = 128 bits
    else:
        m_bytes = bytes.fromhex(m)
        blocks, padding_length = expand_and_split(m_bytes, block_size=16)

    input = Nonce # (variable) blocks: list[bytes]
    processed_blocks = []
    for block in blocks:
        if enc:
            # Encrypt the input and XOR it with the current block
            input = bytes(a ^ b for a, b in zip(input, block))
            encrypted = encrypt(input, k)
            processed_blocks.append(encrypted)
            input = encrypted
        else:
            # Decrypt the input and XOR it with the current block
            decrypted = decrypt(block, k)
            processed_blocks.append(bytes(a ^ b for a, b in zip(input, decrypted)))
            input = block

    # Combine the processed blocks into a single result
    result = b"".join(processed_blocks)

    if enc:
        # Return as hex string for easier storage/transmission
        return result.hex(), padding_length
    else:
        # Remove padding and decode back to the original string
        result = unpad_message(result, padding_length)
        return result.decode("utf-8", errors="ignore")
```

در نهایت به ازای هر مد یک تابع test طراحی شده که خروجی آن در ادامه آورده شده است.

```
[Running] python -u "c:\Users\Ali_Taba\Desktop\Project\CR\CR_PROJECT_PHASE(1)\p1\coco128.py"
EBC:
Encrypted (hex): aa52b7c13bd71ba6f56660c9e2244f55641fb51692ea43005fc4f9148d83902a
Decrypted: Hello, this is a secret message!
OFB:
Ciphertext (Hex): 236742fb18b13c8877d8cce6e115b7a364650ab957ccf214deda36fd045108d4
Decrypted Message: Hello, this is a test message!
CTR:
Ciphertext (Hex): c38b294e37624b8cde83447bde6e3311
Decrypted message: Hello, World!
CBC:
Ciphertext (Hex): 002e7c772614836e928dbcbf8f5c812a
Decrypted message: Hello, CBC Mode!
```